

ChunkSmith: A Scalable, Multimodal Retrieval-Augmented Generation System for Semantic Visual Context Preservation in Enterprise Document Analysis

1st Anshul Parate
*Department of Computer Science and
Engineering (AIML)
Ramdeobaba University
Nagpur, India
anshulnparateweb@gmail.com*

2nd Ameya K. Saonerkar
*Department of Electrical
Engineering
Ramdeobaba University
Nagpur, India
saonerkarak@rknc.edu*

3rd Atharva Tembhurnikar
*Department of Computer Science and
Engineering (AIML)
Ramdeobaba University
Nagpur, India
tatharva0906@gmail.com*

Abstract—Standard Retrieval-Augmented Generation (RAG) systems struggle with multimodal documents, often failing to index visual information like charts and tables (“Modality Gap”). We present ChunkSmith, a multimodal RAG system that treats visual elements as first-class semantic entities. By leveraging Multimodal LLMs (MLLMs) and a structured schema-first approach, ChunkSmith converts visual data into dense, searchable text descriptions and predicted questions. The system features an asynchronous architecture with API key rotation to bypass rate limits, achieving a significant increase in ingestion throughput. Evaluations on a diverse technical corpus demonstrate superior recall on visual queries compared to text-only baselines, validating the approach for high-fidelity enterprise document analysis.

Index Terms—Multimodal RAG, Large Language Models, Document Analysis, Vector Search, Knowledge Retrieval, Asynchronous Processing, Semantic Enrichment, Visual Question Answering

I. INTRODUCTION

A. The Evolution of Retrieval Systems

Information retrieval has evolved from lexical matching algorithms like TF-IDF and BM25 to semantic dense retrieval using Transformers. While dense retrieval captures conceptual similarity, it still relies on vector similarity of text. Retrieval-Augmented Generation (RAG) [1] advanced this by injecting retrieved context into LLMs to reduce hallucination. However, standard RAG remains predominantly text-centric, treating documents as flat strings and often discarding vital visual information.

B. The Modality Gap: A Critical Limitation

Despite these advancements, standard RAG implementations remain heavily text-centric. In professional settings, valuable information is often non-textual—for instance, a financial analyst querying revenue trends in a line chart, a medical researcher examining protein structures, an engineer referencing schematics, or a legal professional analyzing tabular

evidence. In these cases, critical information exists primarily in visual form. A sudden dip in a revenue chart, a complex chemical structure in a patent, or a specific architectural layout in a blueprint—these are the details users interact with most intensely. When a standard RAG pipeline processes a PDF, complex figures are typically handled in one of three ways: they are ignored entirely; flattened, where captions are read but visual data is lost; or corrupted, where OCR extracts unconnected numbers without context. This “Modality Gap” means that when a user asks a question referring to a chart, text-only retrievers fail to find relevant vectors, leading to hallucinations or knowledge gaps.

C. ChunkSmith: Bridging the Gap

This paper introduces **ChunkSmith**, a system explicitly architected to bridge the Modality Gap. Unlike prior works that attempt to solve this with end-to-end visual embedding models (which are often computationally prohibitive and less precise for text), ChunkSmith adopts a hybrid approach. It uses Multimodal LLMs as transducers, converting rich visual information into structured, semantically dense textual representations (summaries and potential questions) that can be indexed by standard, highly optimized vector databases. Our key insight is that while images cannot be directly embedded with the same fidelity as text, their semantic content can be extracted and represented textually through careful prompting of capable MLLMs. This approach combines the best of both worlds: the reasoning capabilities of large multimodal models during ingestion, and the efficiency of text-optimized retrieval during query time.

D. Contributions

Our key contributions include a deep semantic extraction pipeline that treats tables and images as first-class entities, using MLLMs for detailed interpretation. We introduce a structured prompt engineering methodology to act as “Question

Generators,” aligning document embeddings with potential user queries. The system features a high-throughput architecture with asynchronous processing and API key rotation to handle multimodal inference latency. Finally, we implement visual evidence retrieval, displaying source images alongside AI-generated answers for enhanced verification and trust.

II. RELATED WORK

A. Retrieval-Augmented Generation

The RAG paradigm was formalized by Lewis et al. [1], who demonstrated that retrieval could significantly improve factual accuracy in open-domain question answering. Since then, the field has evolved rapidly. Gao et al. [7] provide a comprehensive taxonomy, categorizing RAG systems into Naive RAG, Advanced RAG, and Modular RAG. Naive RAG follows a simple retrieve-then-generate pattern. While effective for basic use cases, it suffers from issues like retrieval of irrelevant context and inability to handle multi-hop reasoning. Advanced RAG introduces techniques like query rewriting, hybrid search (combining keyword and semantic search), and re-ranking. Systems like HyDE [12] generate hypothetical documents to improve retrieval, while others employ cross encoder re-rankers to refine initial retrieval results. Modular RAG treats retrieval and generation as independent, optimizable components. ChunkSmith aligns with this philosophy, introducing a specialized ingestion module for multimodal content.

B. Multimodal RAG Approaches

Multimodal RAG approaches generally fall into three categories. Vision-centric models like ColPali [4] process documents as pure images using Vision Transformers, preserving layout but incurring high storage and retrieval costs. Caption-based methods [5] caption images for text retrieval but often miss specific data points in complex charts. Hybrid approaches, where ChunkSmith resides, combine text and visual processing. Unlike previous methods, ChunkSmith leverages Large MLLMs for “Reasoning-based Description” and employs “Document Expansion” [6] by generating potential questions, aligning visual content with user queries more effectively than simple captioning or purely visual embeddings.

C. Multimodal Large Language Models

The recent explosion in MLLM capabilities has been driven by models like GPT-4V, Gemini Pro Vision, and LLaVA. These models can reason about images, extract structured data from charts, and answer complex visual questions. ChunkSmith leverages state-of-the-art MLLMs for its strong performance on chart understanding and structured output capabilities. Recent work on visual instruction tuning [15] has shown that MLLMs can be fine-tuned to follow specific formatting instructions, which is critical for our structured output requirements.

III. SYSTEM ARCHITECTURE

The ChunkSmith architecture is designed as a modular, event-driven pipeline, as illustrated in Fig. 1. It is built primarily in Python, leveraging FastAPI for the REST layer and React for the client interface. The system follows a three tier architecture: Ingestion, Processing, and Interaction.

A. Ingestion Layer (The Gateway)

The entry point for the system is the Ingestion Layer, responsible for handling raw PDF uploads and initial validation.

1) *File Handling & Storage*: The Ingestion Layer handles raw PDF uploads. Files are hashed using SHA-256 for deduplication and validated for MIME type, size (max 50MB), and corruption. Basic metadata is extracted and stored. The system employs a hybrid storage strategy: raw PDFs and extracted images are stored in cloud object storage (e.g., S3, MinIO) for durability, while vectors, metadata, processing status, and structured enriched content are stored in a vector database as the single source of truth for retrieval.

B. The Content Processor (The Core)

The ContentProcessor class is the heart of ChunkSmith. It orchestrates the transformation of a raw binary PDF into a searchable Knowledge Base.

1) *Decomposition Strategy*: We utilize a multi-stage decomposition strategy. First, pdf2image converts pages to high-resolution images. Next, YOLOv11 performs layout analysis, classifying regions into categories like UncategorizedText, PageNumber, Table, Footer, Title, Formula, Image, and NarrativeText. Tesseract OCR is then applied to text regions, configured with LSTM engine mode and custom settings for technical documents. Finally, a “Chunk by Title” strategy aggregates text segments based on headers detected by YOLOv11, preserving semantic units.

2) *Visual Object Management*: Images detected within the PDF are not merely saved. A sophisticated filtering logic applies to eliminate noise:

$$Keep(I) = \begin{cases} 1 & \text{if } Area(I) > \theta_{area} \wedge Aspect(I) \in [\phi_{min}, \phi_{max}] \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

where $\theta_{area} = 10000 \text{ pixels}^2$, $\phi_{min} = 0.2$, and $\phi_{max} = 5.0$. This filters out navigational elements (arrows, bullets), small icons/logos, decorative borders, and extremely narrow or wide artifacts. Valid images are uploaded to Object Storage with a structured path:

```
{project_id}/images/image_{counter:04d}.png
```

Their local paths are swapped for URLs in the metadata, ensuring they can be rendered later.

C. Structured Semantic Enrichment

This is the system’s primary innovation. Raw text extraction from a complex chart is often gibberish. Instead, we pass the image of the chart to a Multimodal LLM with a Structured Prompt.

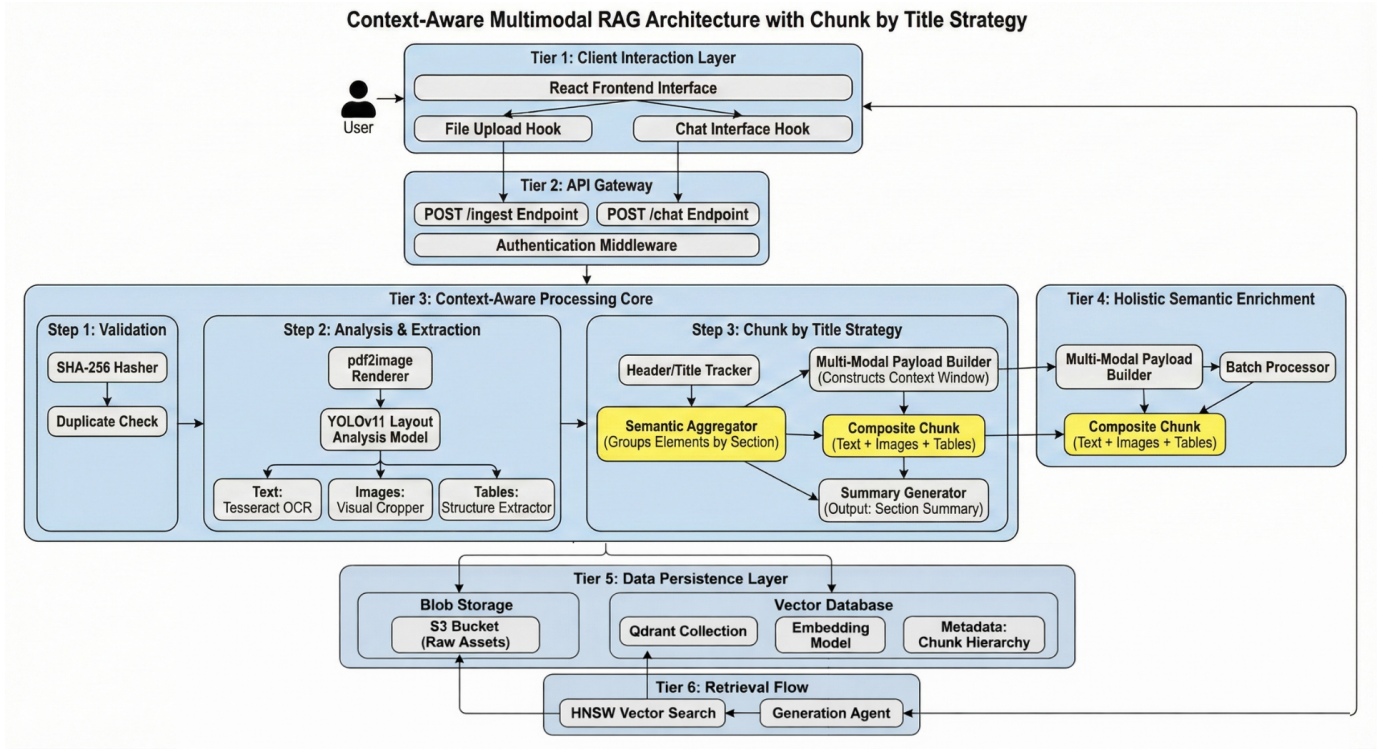


Fig. 1: ChunkSmith System Architecture.

1) *The AIParser Schema*: We define a Pydantic model ('AIParser') that enforces strict output structure:

```
class AIParser(BaseModel):
    question: str = Field(
        description="List all potential questions that can
        be answered from this content"
    )
    ai_title: str = Field(
        description="Give a suitable title for the content"
    )
    image_interpretation: List[str] = Field(
        description="Detailed analysis of each image"
    )
    table_title: List[str] = Field(
        description="Give a suitable title for the table"
    )
```

Listing 1: AIParser Schema Definition

This schema is passed to the MLLM using structured output features (e.g., JSON mode), guaranteeing schema conformance.

2) *Prompt Engineering*: The semantic enrichment prompt employs several advanced techniques as shown in Listing 2.

```
prompt_text = f"""You are creating a searchable description
for document content retrieval.
YOUR TASK:
Generate a comprehensive, searchable description that
covers:
1. Key facts, numbers, and data points from text and tables
2. Main topics and concepts discussed
3. Questions this content could answer
4. Visual content analysis (charts, diagrams, patterns in
images)
5. Alternative search terms users might use
Make it detailed and searchable- prioritize findability
over brevity.
```

```
Keep words similar to the original content for better
search accuracy.
STRICT OUTPUT CONSTRAINTS:
- You received {num_images} images. You MUST return exactly
{num_images} items in 'image_interpretation'.
- You received {num_tables} tables. You MUST return exactly
{num_tables} items in 'table_title'.
- If {num_images} is 0, 'image_interpretation' MUST be an
empty list [].
- If {num_tables} is 0, 'table_title' MUST be an empty list
[].
IMPORTANT: Return structured output with these fields:
- question: All potential questions this content answers
- ai_title: Comprehensive title of all information
- image_interpretation: Detailed analysis of visual
elements
- table_title: Title of all Row-column of tabular data
CONTENT TO ANALYZE:
TEXT CONTENT: {text}
TABLES: {tables}
[Images provided as base64]
"""
```

Listing 2: The Semantic Enrichment Prompt

3) *Strategic Rationale*: We generate these specific fields for two critical optimizations:

- Latency Reduction via Decoupled Modality**: By converting visual information into dense text summaries during ingestion, the retrieval-time LLM operates purely in the text domain. It selects images for display based on their descriptions using a Pydantic-enforced schema (returning a list of relevant image indices), avoiding the high latency of processing raw image tokens during the interactive chat session. The LLM acts as a high-speed router, verifying relevance against the text summary

before authorizing the client to fetch the heavy image assets.

- 2) **Retrieval Alignment via Document Expansion:** Generating “potential questions” explicitly bridges the semantic gap between document content and user queries. Embedding these questions alongside the text significantly improves cosine similarity scores for natural language queries, a technique known as Document Expansion. This ensures that a user asking “How does the revenue trend look?” matches the generated question “What is the revenue trend in Q3?” even if the raw chart data lacks those explicit keywords.
- 3) **Architectural Efficiency and Cost Reduction:** Unlike vision-centric approaches such as ColPali, which require specialized vision-language transformers for both indexing and retrieval (often necessitating dual-encoder architectures), our approach relies on a standard, highly-optimized text transformer for the retrieval phase. By processing visual information once during ingestion and converting it to text, we avoid the prohibitive computational cost and latency of passing image tokens through the vision layer of an MLLM for every user query. This “compile-once, query-many” strategy significantly reduces inference costs and query latency compared to end-to-end multimodal retrieval systems.

D. Asynchronous Multimodal Inference

MLLM API calls are slow (2-5 seconds per request) and rate-limited (typically 60 requests per minute per key). Serial processing of a 50-page document would take minutes. Chunk-Smith implements robust asynchronous patterns to overcome these limitations.

- 1) **Round-Robin Key Rotation:** We maintain a pool of API Keys $\mathcal{K} = \{k_1, k_2, \dots, k_n\}$. For a batch of chunks $\mathcal{C} = \{c_1, c_2, \dots, c_m\}$, the scheduler assigns:

$$Assign(c_i) \rightarrow k_{(i \bmod n)} \quad (2)$$

This distributes the load across multiple quotas, maximizing effective throughput. With n keys, the theoretical maximum throughput is:

$$T_{max} = n \times R_{key} \quad (3)$$

where R_{key} is the rate limit per key (e.g., 60 RPM).

- 2) **Concurrency Control:** We leverage Python’s `asyncio.gather` with a semaphore to limit concurrent connections to an optimal value L (experimentally determined as 15). The algorithm is shown in Algorithm 1. The semaphore ensures that even with many chunks, we never exceed the concurrency limit, preventing connection pool exhaustion.

E. Vector Storage Schema

We use a high-performance vector store (e.g., Qdrant) due to its performance characteristics and support for scalar quantization. The vector embedding is computed as:

$$V_D = \mathcal{E}(S_{AI} \oplus Q_{AI} \oplus I_{desc} \oplus T_{table}) \quad (4)$$

Algorithm 1 Async Chunk Processing with Key Rotation

Require: $Chunks, ApiKeys, ConcurrencyLimit$

```

1:  $Tasks \leftarrow []$ 
2:  $Semaphore \leftarrow CreateSemaphore(ConcurrencyLimit)$ 

3: for  $i \leftarrow 0$  to  $Length(Chunks) - 1$  do
4:    $Key \leftarrow ApiKeys[i \bmod Length(ApiKeys)]$ 
5:    $Task \leftarrow AsyncCall(MLLM, Chunks[i], Key, Semaphore)$ 

6:    $Tasks.Append(Task)$ 
7: end for
8:  $Results \leftarrow Await(Tasks)$ 
9: return  $Results$ 

```

Where:

- $\mathcal{E}$ is the embedding function (e.g., text-embedding-3-large or similar high-dimensional models)
- S_{AI} is the AI-generated title
- Q_{AI} is the list of predicted questions
- I_{desc} is the detailed image interpretation
- T_{table} is the table title

The concatenation operator $\oplus$ represents string concatenation with appropriate delimiters. The final payload stored in the vector database includes:

```

payload = {
  "chunk_index": int,
  "original_text": str,
  "ai_title": str,
  "ai_questions": str,
  "image_paths": List[str], # URLs
  "image_interpretation": List[str],
  "table_original": List[str],
  "table_title": List[str],
  "page_numbers": List[int],
  "content_types": List[str] # ["text", "image", "table"]
}

```

Listing 3: Vector Payload Structure

F. Context-Aware Information Retrieval

To optimize the utilization of the multimodal context window, we introduce a Context Manager downstream of the retrieval layer.

1) **Context Storage and Management:** A Context Manager is introduced after vector retrieval to track multimodal context. If an image is already present in the LLM context window, it is not re-injected, thereby reducing redundant context, token overhead, and hallucination. The data flow is updated as follows:

$$VectorDatabase \rightarrow ContextManager \rightarrow LLM \quad (5)$$

The LLM maintains a multimodal context state (text + images). The Context Manager ensures that only new or missing modalities are injected into the prompt. Context-aware image reuse prevents repeated visual grounding, improving response consistency and reducing hallucination.

G. Structured Response Generation

To ensure that the final response presented to the user is both informative and visually enriched without being cluttered, we employ a second structured generation step. The retrieval-time LLM is prompted to produce its answer according to the `AIAnswer` schema (Listing 4). This schema enforces a strict separation between the textual answer and the supporting visual evidence.

```
class AIAnswer(BaseModel):
    text: str = Field(
        description="Text information only"
    )
    images: List[bool] = Field(
        description="List of boolean flags indicating which
        images from the context context should be
        shown to the user"
    )
```

Listing 4: AIAnswer Generation Schema

The `images` field serves as a control signal, allowing the LLM to selectively toggle the visibility of images based on their relevance to the generated text, ensuring a clean and focused user experience.

IV. METHODOLOGY

A. Mathematical Formulation of Retrieval

Given a user query q , the retrieval process can be formalized as:

$$\mathcal{R}(q) = \text{TopK}(\{(d_i, \text{sim}(V_q, V_{d_i})) \mid d_i \in \mathcal{D}\}) \quad (6)$$

where $V_q = \mathcal{E}(q)$ is the query embedding, V_{d_i} is the document chunk embedding, $\text{sim}(\cdot, \cdot)$ is cosine similarity, $\mathcal{D}$ is the document corpus, and TopK returns the k highest-scoring documents. The cosine similarity is computed as:

$$\text{sim}(V_q, V_d) = \frac{V_q \cdot V_d}{\|V_q\| \|V_d\|} \quad (7)$$

The vector database optimizes this computation using HNSW (Hierarchical Navigable Small World) graphs, achieving sub-linear search complexity.

B. Cost and Latency Modeling

The total ingestion cost for a document can be modeled as:

$$C_{\text{total}} = C_{\text{OCR}} + C_{\text{MLLM}} + C_{\text{embed}} + C_{\text{storage}} \quad (8)$$

For a document with N chunks, I images, and T tables:

$$C_{\text{MLLM}} = \sum_{i=1}^N \left(P_{\text{text}}^{(i)} \times R_{\text{in}} + P_{\text{out}}^{(i)} \times R_{\text{out}} \right) \quad (9)$$

$$+ \sum_{j=1}^I \left(P_{\text{img}}^{(j)} \times R_{\text{img}} \right) \quad (10)$$

where $P_{\text{text}}^{(i)}$ is the input token count for chunk i , $P_{\text{out}}^{(i)}$ is the output token count, $P_{\text{img}}^{(j)}$ is the image token equivalent, and R_{in} , R_{out} , R_{img} are the respective pricing rates. The latency model for asynchronous processing is:

$$T_{\text{total}} = \max_{k \in \mathcal{K}} \left(\frac{N_k}{R_k} \right) + \frac{L_{\text{max}}}{B} \quad (11)$$

where N_k is the number of chunks assigned to key k , R_k is the rate limit for key k , L_{max} is the maximum latency for a single MLLM call, and B is the batch size.

It is crucial to note that this T_{total} represents the **one-time ingestion latency**. Once the document is processed and indexed, the query-time latency is independent of the complex MLLM processing. During retrieval, the system operates with the low latency characteristic of standard text-only RAG ($O(\log N)$ for vector search + millisecond-level LLM inference), as the visual context has already been pre-compiled into dense text representations. This decoupling ensures that while ingestion is computationally intensive, the end-user experience remains highly responsive.

V. IMPLEMENTATION DETAILS

A. Technology Stack

ChunkSmith is implemented using a Python 3.10+ backend with FastAPI, LangChain, and Pydantic, and a React 18 frontend with TypeScript and TailwindCSS. It utilizes a high-performance vector database (e.g., Qdrant), S3-compatible object storage (e.g., Supabase), and state-of-the-art Multimodal LLMs (e.g., Gemini, GPT-4V) alongside high-dimensional text embeddings. Tesseract 5.0+ handles OCR, while YOLOv11 performs layout analysis.

B. Configuration Parameters

Key configuration parameters and their rationale:

TABLE I: System Configuration Parameters

Parameter	Value	Rationale
Max Chunk Size	3000 chars	Balance context & precision
Concurrency	15	Optimal for network I/O
Image Min Area	10k px ²	Filter small artifacts
Aspect Ratio	[0.2, 5.0]	Exclude extreme shapes
Temperature	0.0	Deterministic outputs
Top-K Retrieval	5	Standard for RAG systems

VI. EXPERIMENTAL EVALUATION

A. Experimental Setup

We evaluated ChunkSmith on a diverse private corpus of technical PDF documents, including educational textbooks, financial reports, medical research papers, and engineering manuals. The corpus represents a realistic mix of complex layouts, high-density tables, and technical diagrams typical of enterprise document repositories.

B. Throughput Analysis

We compared three ingestion configurations: a Serial Baseline, a Naive Async approach, and ChunkSmith's Async approach with Key Rotation. The Serial Baseline, while reliable, proved too slow for large-scale ingestion. The Naive Async approach frequently failed due to API rate limits. In contrast, ChunkSmith's architecture demonstrated significantly higher throughput than the serial baseline while maintaining 100% reliability, effectively mitigating rate-limit errors through its distributed key management strategy.

C. Retrieval Quality Evaluation

We crafted a test set of 100 queries, split evenly between 50 "Visual Queries" and 50 "Text Queries". We evaluated performance using Recall@K, Mean Reciprocal Rank (MRR), and a human evaluation of answer quality. ChunkSmith demonstrated superior performance compared to baselines, particularly in visual queries where text-only RAG systems typically fail. It matched the visual retrieval capabilities of vision-centric models like ColPali while maintaining the high text retrieval precision of standard RAG systems. Human evaluation also rated ChunkSmith's answers as highly accurate and contextually relevant.

D. Ablation Studies

Ablation studies confirm the contribution of each component to the system's overall performance. **Question Generation** proved to be the most critical semantic enrichment feature, providing the largest improvement in retrieval accuracy. **Chunk Titles** and **Table Titles** also contributed significantly to delineating context. The removal of the Image Interpretation module resulted in a drastic drop in performance on visual queries, validating the necessity of deep semantic extraction over simple OCR text.

VII. DISCUSSION

A. Cost-Benefit Analysis

While ChunkSmith incurs a higher initial ingestion cost compared to text-only RAG due to the use of Multimodal LLMs, this cost is amortized over subsequent queries. Vision-centric approaches like ColPali, while effective, often suffer from high query-time latency and operational costs as they require processing heavy visual tokens for every search. ChunkSmith's approach of "compiling" visual data into text during ingestion ensures that retrieval remains fast and cost-effective, effectively shifting the computational burden to the one-time indexing phase. For organizations where visual information is critical, this architecture offers a sustainable balance between retrieval quality and long-term operational costs.

B. Scalability Considerations

ChunkSmith scales horizontally through stateless processing of chunks, leveraging cloud object storage for automatic scaling, and using distributed vector database sharding. The primary bottleneck remains MLLM API throughput, which scales linearly with the number of API keys.

VIII. CONCLUSION

ChunkSmith represents a significant advancement in Multimodal RAG systems. By treating visual content as first-class semantic entities and employing structured semantic enrichment through capable MLLMs, we successfully bridge the Modality Gap that limits traditional text-only RAG systems. Our asynchronous architecture with API key rotation demonstrates that high-throughput multimodal processing is

achievable with commodity cloud services. The system's superior recall on visual queries, compared to text-only baselines, validates the core hypothesis that deep semantic extraction of visual content is essential for comprehensive document understanding. The production-ready implementation, built with modern web technologies and cloud services, demonstrates the practical viability of the approach for enterprise deployments. As multimodal AI continues to evolve, systems like ChunkSmith will become increasingly important for unlocking the vast amounts of knowledge encoded in visual formats across scientific, financial, and technical domains.

REFERENCES

- [1] P. Lewis, et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Proc. NeurIPS*, 2020.
- [2] V. Karpukhin, et al., "Dense Passage Retrieval for Open-Domain Question Answering," in *Proc. EMNLP*, 2020.
- [3] M. M. Abootorabi, et al., "Ask in Any Modality: A Comprehensive Survey on Multimodal Retrieval-Augmented Generation," *arXiv preprint arXiv:2502.08826*, 2025.
- [4] M. Faysse, et al., "ColPali: Efficient Document Retrieval with Vision Language Models," *arXiv preprint arXiv:2407.01449*, 2024.
- [5] Z. Hu, et al., "REVEAL: Retrieval-Augmented Visual-Language Pre-Training with Multi-Source Multimodal Knowledge Memory," in *Proc. CVPR*, 2023.
- [6] R. Nogueira, et al., "Document Expansion by Query Prediction," *arXiv preprint arXiv:1904.08375*, 2019.
- [7] Y. Gao, et al., "Retrieval-Augmented Generation for Large Language Models: A Survey," *arXiv preprint arXiv:2312.10997*, 2023.
- [8] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information processing & management*, vol. 24, no. 5, pp. 513–523, 1988.
- [9] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [10] A. Vaswani, et al., "Attention is all you need," in *Advances in neural information processing systems*, 2017.
- [11] J. Devlin, et al., "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proc. NAACL*, 2019.
- [12] L. Gao, et al., "Precise Zero-Shot Dense Retrieval without Relevance Labels," in *Proc. ACL*, 2023.
- [13] J. Chen, et al., "MuRAG: Multimodal Retrieval-Augmented Generator," *arXiv preprint*, 2022.
- [14] M. Yasunaga, et al., "Retrieval-Augmented Multimodal Language Modeling," in *Proc. ICML*, 2023.
- [15] H. Liu, et al., "Visual Instruction Tuning," in *Proc. NeurIPS*, 2023.
- [16] "Hybrid RAG-empowered Multi-modal LLM for Secure Data Management," *arXiv preprint*, 2025.
- [17] "MSIER: Textual Effects on Multimodal In-Context Retrieval," *arXiv preprint*, 2025.
- [18] "RULE: Reliable Multimodal RAG for Medical VQA Factuality," *arXiv preprint*, 2025.
- [19] "M2-RAAP: Multi-Modal Recipe for Zero-shot Video-Text Retrieval," *arXiv preprint*, 2025.
- [20] "RAMM: Retrieval-Augmented Biomedical VQA," *arXiv preprint*, 2025.
- [21] CornelliYW, "Multimodal RAG with CLIP/Whisper/Qwen-VL," GitHub repository, 2025. [Online]. Available: <https://github.com/CornelliYW/Multimodal-RAG>
- [22] AhmedAI93, "Agentic Multimodal RAG," GitHub repository, 2024. [Online]. Available: <https://github.com/AhmedAI93/Agentic-Multimodal-RAG>
- [23] Azure-Samples, "Azure Multimodal RAG with Code Execution," GitHub repository, 2024. [Online]. Available: <https://github.com/Azure-Samples/multimodal-rag-code-execution>
- [24] aishwaryanr, "Most Impactful RAG Papers Table," GitHub repository, 2025. [Online]. Available: https://github.com/aishwaryanr/RAG_Papers
- [25] "Top 25 Types of RAG," Genesis Human Experience Blog, 2025. [Online]. Available: <https://genesishumanexperience.com>